A02c · When a Node Has Two Pointers — Practice A2 · 9618 §19.1

Name: _____ Due: _____

This topic is examined **twice** — drawn and tabled on Paper 3, typed on Paper 4, so this homework has both. **Questions 1–6 are written, on this sheet. Questions 7–10 are code, in `A02c_Practice.java`** (listed at the bottom). One set of numbers across both halves. Throughout, an ordered binary tree stores **integers** in a 2D array: **column 0 holds the left pointer, column 1 the data, column 2 the right pointer**; `−1` is the null pointer, each new value is stored at the next free row, a smaller value goes **left**, and a value **equal to a node's data goes right** (it follows "otherwise"). Nothing on this sheet ever deletes a node.

1. Read it. A binary tree is stored in the array `ArrayNodes` below. [3]

Index	LeftPtr	Data	RightPtr
0	1	40	2
1	3	23	4
2	−1	55	−1
3	−1	8	−1
4	−1	31	−1

```
RootPointer = 0
FreeNode    = 5
```

(a) State the value at the **root** of the tree, and how the table tells you. [1]

(b) Write the output of an **in-order traversal** (left subtree · node · right subtree, at every node). [2]

2. Grow one. The values `2, 7, 1, 8, 2, 8` — the first six digits of *e* — are inserted into an empty tree, in that order. **Draw the tree**: circles for nodes, arrows for pointers. Remember the tie-break stated at the top — an equal value goes **right**. [4]

3. The paper's table. The same six inserts, `2, 7, 1, 8, 2, 8`, as the array sees them. The `Data` column is done for you — each value sits at the row that was free when it arrived. **Complete every pointer cell, and both boxes.** [4]

Index	LeftPtr	Data	RightPtr
0		2	
1		7	
2		1	
3		8	
4		2	
5		8	

`RootPointer =`

`FreeNode =`

Your drawing in question 2 and this table are the **same tree** — use one to check the other.

4. Complete it. *(Adapted from Nov 2024 Paper 31 — the paper stores words; here the data are integers.)* A different program keeps the nodes in a 1D array of **records** called `BinTree`: each record has the fields `LeftPtr`, `Data` and `RightPtr`, and `RootPtr` points at the root record. The function returns the **index** of the record holding `Item`, or the null pointer `-1` if it is not found. Fill in the four gaps. [4]

```
FUNCTION SearchTree(Item : INTEGER) ① _____________________
   NowPtr ← ② _____________________
   WHILE NowPtr <> -1
      IF ③ _____________________ THEN
         NowPtr ← BinTree[NowPtr].LeftPtr
      ELSE
         IF BinTree[NowPtr].Data < Item THEN
            NowPtr ← ④ _____________________
         ELSE
            RETURN NowPtr
         ENDIF
      ENDIF
   ENDWHILE
   RETURN NowPtr
ENDFUNCTION
```

5. The order that ruins it. A program loads 1000 customer IDs into the tree — and the IDs arrive **already sorted, ascending**. [2]

(a) Describe the shape of the tree this builds. [1]

(b) State the consequence for the speed of **`SearchTree`**, compared with a well-shaped tree. [1]

6. The counter that used to be a chain. The linked list you studied kept **`FirstEmpty`** — a whole **chain** of free nodes. This tree keeps only **`FreeNode`**, a single counter that only ever counts up. State the property of this tree's use that makes a counter enough, and name the operation that would force the free chain back. [2]

💻 **QUESTIONS 7–10 — ON THE COMPUTER, NOT ON THIS SHEET.** Open **`A02c_Practice.java`**. It contains a short unmarked warm-up and then these questions. The file **tests itself** — including against the exact output Cambridge printed in the November 2021 mark scheme, and a test with no answer key: random trees whose in-order walk must come out sorted.

7	**`PrintAll()`** — print every row of the array in array order: left pointer, data, right pointer	[2]
8	**`AddNode()`** — the insert walk: store at **`FreeNode`**, attach through the **parent's** pointer, FALSE when full	[6]
9	**`InOrder()`** — the lesson's cliffhanger, typed: left, self, right — three lines and one of them calls itself	[5]
10	🍦 **`SearchTree()`** — question 4, as real code — optional, not marked	—

To hand in: rename the file to **`A02c_YourName.java`** and submit it. Change nothing inside — the class is not **`public`**, so Java does not mind what the file is called. · **Written 19 marks + code 13 marks = 32.**

🕳️ **Want to dig?** Question 5's ruined tree is not the end of the story. There is a tree that **refuses** to become a stick: after every insert it measures its two sides, and if they differ by more than one, it *rotates* — three pointers re-wired, order preserved, balance restored. It has been known since 1962, and it is waiting for you as the 🍨 third scoop at the bottom of **`challenge_A02c_BinaryTree`**. Fair warning: it is the hardest thing in this unit, and the first true taste of next lesson.

Additional space. If you need more room for any answer, continue here and write the question number in the margin beside it.